Repair Bst

AlgoExpert · Hard · Binary Search Trees

The problem

- Exactly two nodes' values were swapped; swap them back in place.
- Iterative in-order finds the two out-of-order nodes.

Intuition

- An in-order traversal of a BST is sorted; the two swapped nodes show up as order violations.

Approach

- 1. In-order traverse tracking the previous node
- 2. The first violation gives the first node
- 3. The last violation gives the second
- 4. Swap their values

Java solution

```java
public class RepairBst {
    public static BST repairBst(BST tree) {
        BST first = null, second = null, prev = null, cur = tree;
        ArrayDeque<BST> st = new ArrayDeque<>();
        while (cur != null || !st.isEmpty()) {
            while (cur != null) { st.push(cur); cur = cur.left; }
            cur = st.pop();
            if (prev != null && prev.value > cur.value) { if (first == null) first = prev; second = cur; }
            prev = cur;
            cur = cur.right;
        }
        int t = first.value; first.value = second.value; second.value = t;
        return tree;
    }
}
```

Complexity

- Time: O(n)
- Space: O(h)

Recap

- An in-order traversal of a BST is sorted; the two swapped nodes show up as order violations.
- Complexity: O(n)